

CSI243: Functional programming

A Haskell function takes one or more arguments and gives a single result

- **Function defined using:-**
 - **an equation that give the function name**
 - **a name for each of its arguments**
 - **a function body that specifies how the result is computed in terms of its arguments**
- **A function is applied to the actual arguments, the result is obtained by substituting the formal parameters with them**

CSI243

Functions

- **Functional programming focuses on the model of the problem domain focusing on:-**
 - **Collection of data types**
 - **User defined functions and types**
- **A function should do one and only one logical task (cohesion in SE)**
- **Functions focus on what is to be computed, not how it should be computed**
- **Composition of functions and recursion**

Functions

CSI243

- **Composition operator, denoted by $\circ$, is the expression by $f \circ g$, is itself a function**
- **Application of the function f on the argument (which is a function itself) is the composition $f \circ g$ or more formally**
 - $\circ :: ((A \rightarrow C), (B \rightarrow A)) \rightarrow (B \rightarrow C)$
 - $(f \circ g) x = f (g x)$
- **The type of $\circ$ requires functions of compatible types**
- **Function's implementation is derived from its specification**

CSI243

Functions

- **The input set I comprises of both the invalid subset I_e and the valid subset I_v**
- **The output set O comprises of both the erroneous subset O_e and valid subset O_v**

$$I = I_v \cup I_e \text{ where } I_v = I \setminus I_e$$
$$O = O_v \cup O_e \text{ where } O_v = O \setminus O_e$$

- **A function f acts on the arguments from I_v (precondition state) to give output in O_v (postcondition state)**

CSI243

Functions

- **The function f may produce some anomaly even on arguments from I_v , which will result in the output in O_e**
- **The two different type of errors being:-**
 - **Syntax errors**
Deals with errors due to the programming language structure or syntax
 - **Semantics errors**
Deals with errors due to the interpretation or semantics of the program and the programming language

- **Some examples from the two different type of errors include:-**

- **Syntax errors**

```
if (2 = "tree") st1
```

```
else st2;
```

```
int res = total / no;
```

type mismatch

imbalanced parenthesis; ...

- **Semantics errors**

```
int res, total, no;
```

```
res = total / no;
```

list index out of range

division by zero; ...

CSI243

Functions

- **Functional programming vs imperative programming some of the differences:-**
 - **States changes**
 - **Algorithmic**
 - **Side effects**
 - **polymorphism**
- **Haskell programs are high-level ie:-**
 - **functions can be applied to arguments, functions may also be arguments**
 - **Functions don't yield side-effects**
 - **Parallelism is easier to realise**
 - **Polymorphism supported**

CSI243

Haskell program

- **program files have “.hs” extension**
- **filename – meaningful, implied contents**
- **Program has:-**
 - **program text**
 - **comments :-**
 - **Single ones start with “--” or**
 - **Block enclosed in “{ –” and “– }”**
 - **modules:-**
 - **has a name**
 - **will contain a collection of Haskell definitions**
 - **one module per file; name of file corresponds to the module name**

Haskell program **CSI243**

- **Name of functions and their arguments format :-**

`[a-z][a-z | A-Z | _ | ` | 0-9]*`

- **Keywords cannot be used to name functions or their arguments. Some examples of keywords:-**

`type, let, if, import, then, where, ...`

- **Indentation of the definition of the functions helps in readability**
- **List arguments end with `s` in their names to indicate that they may have values. Some examples :-**

`detLen lists = ...`

`avg ints = ...`

CSI243

Haskell program

- **Module definition format :-**

```
module ModName where
```

```
:
```

```
:
```

- **Name of module starts with a capital letter**
- **A module may import definitions from other modules**

```
module ModName where
```

```
    import OtherModule
```

```
:
```

```
:
```

CSI243

Haskell program

- `Prelude` **is the standard library that is loaded when Haskell is started**
- **Standard** `Prelude` **from** `Prelude.hs` **extension.**
- **It gives the function signatures for several tasks**
- **Imported by default into other modules**
- **See the brief** `Prelude` **documentation**
https://wiki.haskell.org/Learn_Haskell_in_10_minutes
<https://ghc.readthedocs.io/en/8.0.1/>
- `GHCI` **commands start with “:”**

CSI243

Haskell program

- **Some of tasks offered in** `Prelude` :-
 - **External program calls** `--- :! oscom`
 - **Load module(s)** `--- :load module`
 - **Help** `--- :?`
 - **give function(s) signatures in module(s)** `--- :browse module`
 - **Setting the environment**
 - **Basic debugging :-**
 - **Setting breakpoints**
 - **Tracing up to a breakpoint**
 - **Display** `--- :show modules`
 - **Show type of expression**
 - **Exit**

CSI243

Haskell program

- **Some of tasks offered in** `Prelude` :-
 - **Comparison (see the polymorphic equality test operation)**
 - **Basic I/O :-**
 - **Simple input**
 - **Simple output**

Basic types

CSI243

- **A type is a collection of related values**
- **Every expression must have a type in functional programming**
- **Give some examples of basic types:-**
 - **Boolean**
 - **Real or float**
- **A function f acting on arguments has a result type, and each argument has a type.**
- **Example**

```
neg :: Bool -> Bool
```

- **function** `neg` **has type** `Bool -> Bool`
- **argument's type is** `Bool`
- **result's type is** `Bool`

Basic types

CSI243

- **Every Haskell expression must have a type**
- ***Type inference* a process of calculating the type of expression prior to evaluating the expression**

- **Type inference**

- $$\frac{f :: A \rightarrow B \quad e :: A}{f \ e :: B}$$

- **Typing rule for function application, which states that if f is a function that maps arguments of type A to results of type B , and e is an expression of type A , then the application of $f \ e$ has type B .**

Basic types

CSI243

- **Boolean type**

- **has the values** `True` **and** `False`
- **Used to test if a given condition holds**
- **May be used to give choices in function definitions by means of guards**
- **Implemented in Haskell as** `Bool`

- **Examples**

```
gone :: Bool -- gone has type Bool
neg  :: Bool -> Bool {- neg has
                      type (Bool -> Bool) -}
```


CSI243

Basic types

- **Example**

- **Given**

- `pasd :: Bool`

- `neg :: Bool -> Bool`

- **Use the typing inference rule to deduce the type of the following:-**

- `1.neg pasd`

- `2.neg 3`

CSI243

Basic types

- **Example**

- **Using the type inference rule**

$$1. \frac{\text{neg} :: \text{Bool} \rightarrow \text{Bool} \quad \text{pasd} :: \text{Bool}}{\text{neg pasd} :: \text{Bool}}$$

Thus the type of `neg pasd` is valid

$$2. \frac{\text{neg} :: \text{Bool} \rightarrow \text{Bool} \quad 3 :: ??}{\text{neg } 3 :: ???}$$

Thus the type of `neg 3` is invalid